

Stack <int> myStack;

myStack.push(10);

Stack <int> temp;

while (!temp.empty())

{

cout << temp.top() << " "

temp.pop();

}

cout << myStack.top();

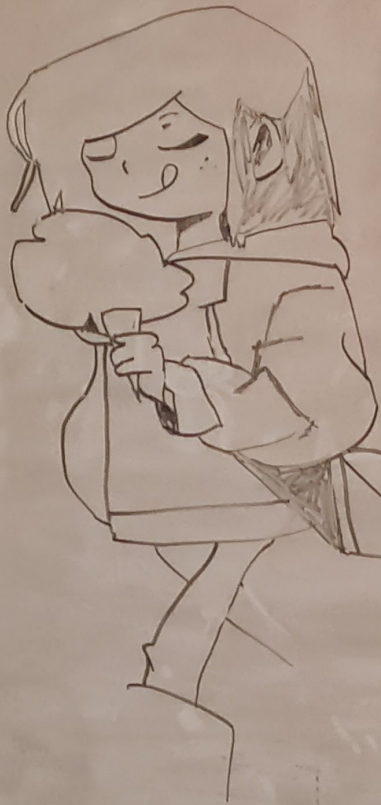
Data Structures

Front = Front + 1

Rear = Rear + 1

Queue DSA

FIFO



Set

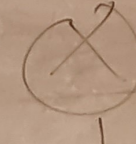
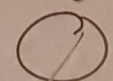
R = -1

F = -1

empty

Front

Queue



Stack <int> myStack

myStack.push(10)

Stack <int> temp

while (!temp.empty())

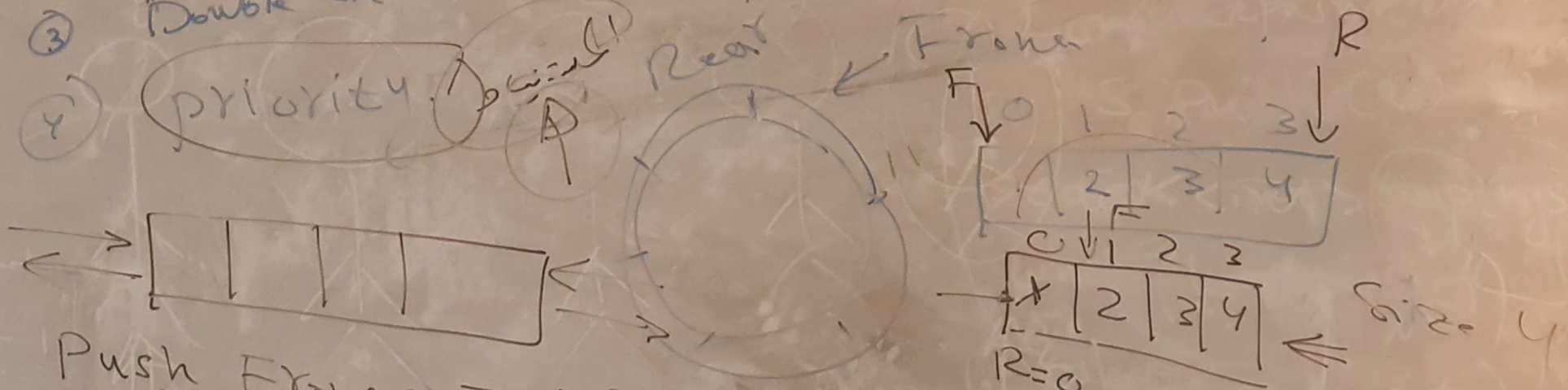
cout << temp.top();
temp.pop();

cout << myStack.top();

Rear
or
back
tail

Data Structures

- ① Simple Queue.
- ② Circular =
- ③ Double ended Queue Deque
- ④ priority Queue



Push_Front
 Push_Back
 Pop_Back
 Push_Back

input end, 2 - del

Data Structure

```
#include <queue>
```

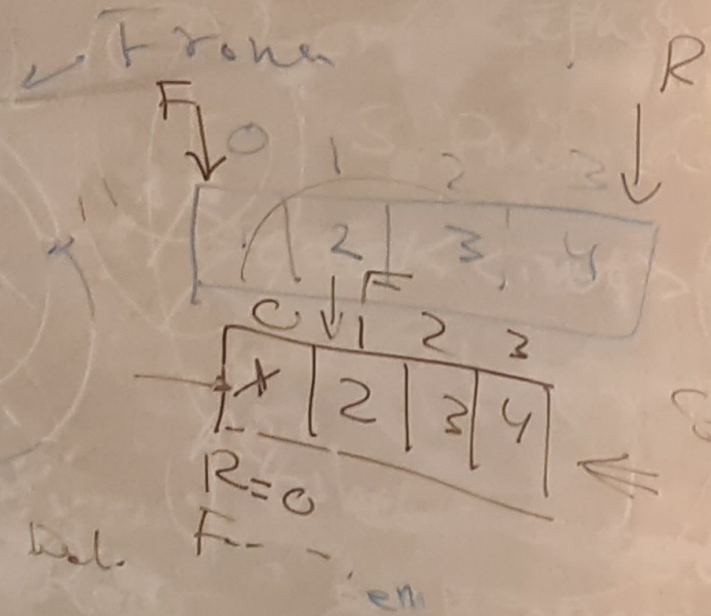
```
queue<int> myQueue;
```

① push()
 enqueue()
 input element

② pop()
 dequeue
 Size()

③ Front()
 rear()
 back()

④ rear()
 back()



Data Structures

①
 Q. Push(10);
 Q. push(20);
 Q. push(30);
 POP();
 POP();

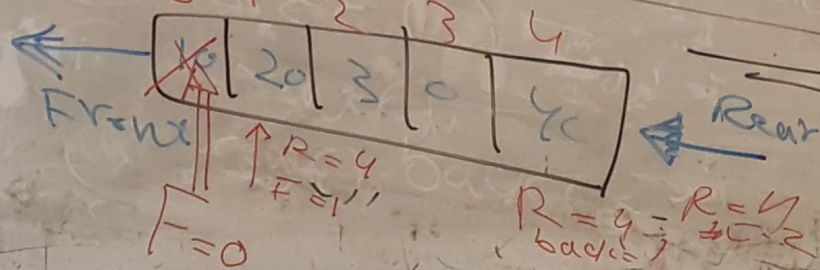
② empty

③ print Front, back

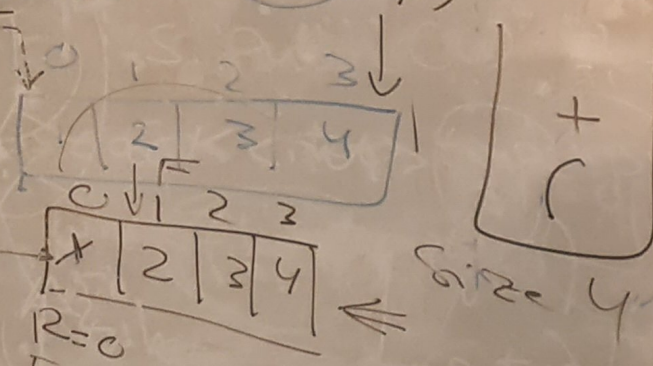
R = -1
 F = -1

② if (empty) * underflow

③ if Full = 1 = max size - 1



* (A+B) * (C/D) while (1)
 (3+4) * (6/2)



overflow queue()